BBR-ES: An Extended-State Optimization for BBR Congestion Control

Zewei Han and Go Hasegawa, *Member, IEEE*

***Abstract*—In recent years, many optimization proposals for TCP BBR have been introduced, but most rely mainly on delay variations and do not fully resolve BBR's limitations in RTT fairness, link utilization, and delay control in networks. This paper proposes BBR with Extended State (BBR-ES), which extends BBR's state machine with a short stabilization state and a trend-based transition mechanism that react to per-flow bandwidth and RTT evolution instead of global delay alone. BBR-ES uses lightweight bandwidth and RTT trend tracking to adjust its sending rate while preserving BBR's model-based design. Experiments on both emulated (Mininet) and real-world Internet paths (Amazon EC2) show that BBR-ES consistently improves RTT fairness and link utilization over BBRv1, BBRv3, and CUBIC while keeping queuing delay moderate and bounded; in most settings, it achieves Jain's fairness index above 0.9 and link utilization above 98%. These results indicate that BBR-ES is a practical candidate for deployment in large-scale content delivery and a useful design reference for future model-based congestion control schemes.**

***Index Terms*—Congestion control algorithm, bottleneck bandwidth and round-trip propagation time (BBR), throughput fairness, round trip time (RTT).**

I. Introduction

In 2016, GOOGLE introduced the Bottleneck Bandwidth and RTT algorithm (BBR) [1], which fundamentally changed congestion control by moving away from packet loss as the primary congestion signal. Instead, BBR dynamically adjusts the sending rate by estimating the bottleneck bandwidth and the queue-free round-trip time in real time, aiming to operate near the optimal bandwidth–delay product (BDP). In single-flow scenarios, BBR adapts well to variations in bandwidth and RTT and avoids the drastic rate fluctuations typically observed in loss-based algorithms. This has led to widespread adoption in video streaming and large-scale file transfer applications such as Amazon and Netflix [2], [3]. Measurement studies report that BBR has quickly become a dominant congestion control algorithm: its deployment rate was around 18% in 2019 with a traffic share exceeding 40%, increasing to 22% by 2023, and it remains one of the most widely used CCAs among the top 20,000 Alexa-ranked websites [4], [5], [6]. However, these statistics mainly reflect BBR's market adoption; in shared bottleneck scenarios with many concurrent flows, significant challenges remain in terms of fairness and delay control.

Many studies [7], [8], [9], [10] have shown that when multiple flows share a bottleneck link, BBR tends to overestimate the available bandwidth and operate away from the ideal BDP. This leads to persistent queue build-up, increased delay, and a slow reaction to changes in network conditions. More critically, when flows have heterogeneous RTTs, severe fairness problems arise: in deep-buffer environments, long-RTT flows tend to occupy a disproportionate share of the bandwidth, squeezing out short-RTT flows that may receive little or almost no throughput. Such RTT unfairness not only affects individual flows but also harms the overall utilization and efficiency of the network.

To address these issues, many BBR variants have been proposed. However, most of them rely on delay variation as the only or primary signal for adjusting the sending rate, congestion window (CWND), or related parameters [8], [11], [12], [13]. These approaches typically adopt a "one-size-fits-all" strategy: when an increase in overall network delay is detected, all flows are forced to reduce their sending rates simultaneously. This uniform control mechanism ignores the fact that different flows contribute very differently to queue build-up. Some flows occupy only a small share of the bandwidth and keep their inflight packets well below the fair level; nevertheless, because they are governed by a global delay metric, they are also forced to slow down. This unnecessary rate reduction further diminishes their bandwidth usage and link utilization, and can even exacerbate fairness problems. In other words, controlling only on a global delay signal scratches the surface of congestion and unfairness and often penalizes flows that do not merit a reduction in rate.

Motivated by these challenges, this paper proposes *BBR with Extended State* (BBR-ES), which aims to improve RTT fairness, link utilization, and delay control in multi-flow environments while preserving BBR's model-based nature. The key novelty of BBR-ES is not to retune BBR's parameters, but to change its control structure in two ways. First, BBR-ES extends the state machine with a short *stabilization state* inserted between `PROBE_RTT` and `PROBE_BW`. This extended state allows each flow to recover from the very low rate of `PROBE_RTT` in a controlled way, using per-flow bandwidth and RTT information instead of a single global delay

Received 27 November 2025; accepted 21 February 2026. Date of publication 27 February 2026; date of current version 5 March 2026. The associate editor coordinating the review of this article and approving it for publication was T. Inoue. *(Corresponding author: Zewei Han.)*

Zewei Han is with the Graduate School of Information Sciences, Tohoku University, Sendai 980-8579, Japan (e-mail: han.zewei.p1@dc.tohoku.ac.jp).

Go Hasegawa is with the Research Institute of Electrical Communication, Tohoku University, Sendai 980-8577, Japan (e-mail: hasegawa@riec.tohoku.ac.jp).

Digital Object Identifier 10.1109/TNSM.2026.3668966

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

signal, and to repair inflight imbalance before returning to steady probing. Second, BBR-ES introduces a *dynamic, trend-based transition mechanism* that monitors recent bandwidth and RTT evolution over a small history window and only adjusts the sending rate when consistent trends indicate congestion or under-utilization. Together, these two mechanisms enable BBR-ES to distinguish flows that heavily contribute to queue build-up from those that do not, and to perform targeted corrections without abandoning BBR's model-based design.

To evaluate the practical applicability of BBR-ES, we conduct experiments both in a controlled emulation environment (Mininet [14]) and on real-world Internet paths using Amazon EC2 instances deployed across multiple geographic regions [15], running Linux kernel version 6.4.0. These experiments cover heterogeneous RTT combinations, various buffer sizes, and different numbers of competing flows, and compare BBR-ES against BBRv1, BBRv3 [16], CUBIC, and several BBR variants.

In this work, we primarily target deployment scenarios where BBR and its variants already dominate the traffic mix, such as large-scale video streaming and bulk-transfer services. Our main goal is to improve RTT fairness and queuing behavior among BBR-family flows sharing a bottleneck, rather than to redesign cross-protocol bandwidth sharing for the entire transport ecosystem. Loss-based schemes such as Reno and CUBIC are included in our evaluation mainly as reference points; a systematic coexistence study between BBR-ES and loss-based or hybrid congestion controllers is left for future work.

The main contributions of this paper are as follows:

1) We develop a mathematical model that analyzes the fundamental causes of BBR's RTT unfairness in multi-flow scenarios and explains why existing delay-based modifications, which rely on global delay metrics, are inherently limited in fully resolving these issues.
2) We propose the BBR-ES algorithm, which introduces an extended stabilization state and a lightweight trend-based transition mechanism driven by per-flow bandwidth and RTT evolution. This design enables BBR-ES to react to congestion in a flow-specific manner, mitigating queue build-up and fairness problems without sacrificing BBR's model-based control.
3) We systematically evaluate BBR-ES using Mininet and Amazon EC2 on a modern Linux kernel, and demonstrate that BBR-ES achieves consistently higher RTT fairness and link utilization than BBRv1, BBRv3, and CUBIC, while maintaining moderate and bounded queuing delay across a wide range of RTTs, buffer sizes, and flow counts.

The remainder of this paper is organized as follows. Section II reviews related work. Section III elucidates the root causes and limitations of BBR's RTT unfairness. Section IV details the design and implementation of BBR-ES. Section V and Section VI present the experimental evaluation results, and Section VII concludes the paper and discusses future research directions.

II. Related Work

This section reviews prior work on congestion control from two complementary perspectives. First, we provide a taxonomy of existing congestion control algorithms, highlighting the conceptual differences that motivate the design of BBR-ES. Second, we discuss the RTT unfairness problem in BBR and analyze why existing solutions—although effective in certain scenarios—remain fundamentally limited.

A. Taxonomy of Congestion Control Algorithms

Existing congestion control algorithms can be broadly categorized into four classes:

(i) **Loss-based algorithms.** Classical TCP variants such as Reno [17] and CUBIC [18] rely on packet loss as the primary congestion signal. Although AIMD-style rate adjustment ensures stability, it suffers from bufferbloat [19] and slow convergence in modern high-speed networks.

(ii) **Delay-based algorithms.** Schemes such as TCP Vegas measure RTT inflation as a proxy for queue buildup. These algorithms react earlier than loss-based variants but are highly sensitive to RTT noise and cross-traffic, making them fragile in heterogeneous environments.

(iii) **Model-based algorithms.** BBR [2], [3] and its variants construct an explicit model of the network using bottleneck bandwidth and propagation delay. This paradigm departs fundamentally from loss- or delay-based control, enabling high utilization and low queuing delay even in deep-buffer environments. As measurement studies [4], [5], [6] show, BBR has been widely deployed in video streaming and large-scale content delivery systems, becoming a major component of today's Internet traffic.

(iv) **Hybrid and auxiliary approaches.** Recent efforts combine multiple congestion signals (loss, delay, or ECN) or leverage machine learning to infer appropriate sending rates [20]. Although these methods can improve fairness or robustness, their deployment cost and complexity often limit real-world adoption.

This taxonomy positions BBR-ES firmly within the *model-based* category, aiming to address an intrinsic limitation of BBR: its treatment of the queue as a *global* signal shared by all flows, regardless of how much each flow actually contributes to congestion.

This taxonomy clarifies where BBR and its variants sit in the broader CCA landscape. In the following subsections, we focus on RTT unfairness in BBR and review existing BBR-based proposals, highlighting why they still fall short under heterogeneous RTTs.

B. BBR and RTT Unfairness

Despite its deployment success, BBR faces persistent fairness issues in multi-flow scenarios, particularly when RTTs differ. Prior analyses [5], [7], [9], [21] show that BBR tends to overestimate the available bandwidth and operate above the optimal BDP. Consequently, long-RTT flows often dominate

the bottleneck queue, as they probe for longer [22] and maintain larger inflight data volumes [11]. Other studies [23], [24], [25], [26] similarly attribute unfairness to RTT-proportional inflight allocation and to structural imbalance in how BBR handles queuing delay.

C. Existing Solutions and Their Limitations

A number of BBR variants aim to mitigate RTT unfairness.

1) Delay-Driven Approaches: BBQ [22] shortens the probing duration based on observed delay inflation, while BBR-E and Delay-aware BBR [11], [12] reduce pacing rates aggressively when queuing delay increases. These approaches improve fairness but often sacrifice link utilization [10], [27]. BBR-A [13] and FE-BBR [28] rely on delay thresholds or combined delay/loss signals, but remain sensitive to shallow buffers or heterogeneous RTTs.

2) Other Mechanisms: Adaptive-BBR [23] dynamically adjusts pacing gain per ACK arrival but still fails under large RTT disparity. BBR-EFRA [29] infers buffer occupancy to adapt cwnd, but becomes unreliable in shallow buffers. ML-based approaches [20] offer fine-grained control but incur high computational and data collection costs.

D. Deep Insight: The Limit of Global Delay-Based Control

Although the above variants differ in mechanics, most share a fundamental limitation: they treat increases in queuing delay as a *global* symptom attributable equally to all flows. However, real multi-flow dynamics are highly asymmetric: - Some flows inject significantly more excess inflight packets and are responsible for most of the queue buildup. - Other flows maintain much smaller inflight volumes and contribute minimally, yet they are penalized equally when global-delay mechanisms reduce the rate of *all* flows.

This symmetric treatment leads to two adverse effects: 1) low-impact flows are unnecessarily throttled, reducing their throughput further; 2) high-impact flows retain structural advantages due to their larger RTT or inflight size.

These limitations motivate the design of BBR-ES, which leverages per-flow bandwidth and RTT trends—rather than global delay inflation—to differentiate congestion contributors and perform *targeted* corrective actions without harming benign flows.

E. Experimental Methodologies

Most prior studies [11], [13], [23], [27], [28], [29] evaluate BBR variants using Mininet or ns-3 [30]. While these environments offer reproducibility, they often use limited flow counts, fixed buffer sizes, and narrow RTT settings, which obscure fairness problems that emerge only under broader heterogeneity. Only a few works [16], [31] include real-world validation, leaving uncertainty regarding how these algorithms behave in dynamic Internet environments.

F. Differences From Our Prior Work

Our earlier work [8] relies solely on instantaneous delay inflation and only modifies the PROBE_BW phase. It does not consider bandwidth/RTT trends, provides no convergence mechanism, and treats delay as a global signal shared by all flows. In contrast, BBR-ES introduces a new extended state, trend-based congestion detection, and per-flow inflight correction. These mechanisms fundamentally change BBR's state dynamics and address limitations that our previous delay-based heuristic could not resolve.

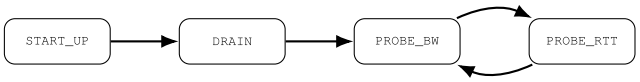

Fig. 1. Transition diagram of BBRv1.

III. BBR Unfairness Analysis

A. BBR Overview

The rate adjustment of BBR is based on two parameters: Bottleneck Bandwidth ($BtlBw$) and Minimum RTT (min_rtt). $BtlBw$ refers to the maximum bandwidth that the link can provide. It is estimated by continuously observing the arrival rate of ACK. BBR will slide the window to observe the "maximum delivery rate in the recent rounds of RTT" as the upper limit of bandwidth. min_rtt refers to the shortest RTT measured when the network is not congested and there is no queuing. The core algorithm of BBR consists of four states, and its state transition diagram is shown as Figure 1:

- `START_UP`: Rapidly increase the sending rate to detect the available bandwidth;
- `DRAIN`: Reduce the sending rate to empty the queues accumulated in the `START_UP`;
- `PROBE_BW`: Periodically adjusts the pacing gain around the estimated bandwidth to maintain high utilization;
- `PROBE_RTT`: Periodically enters a minimum-inflight state for about 200 ms (or one RTT) to refresh the min_rtt estimate.

In each state, the sender determines the packet pacing rate $x(t)$ and the congestion window size $cwnd(t)$ at the time t when the sender receives an ACK packet with Equations (1) and (2), respectively.

$$x(t) = g_p \cdot BtlBw(t) \tag{1}$$

$$cwnd(t) = g_w \cdot BtlBw(t) \cdot min_rtt(t) \tag{2}$$

Among them, the steady state of BBR mainly switches between the states of `PROBE_BW` and `PROBE_RTT`. g_p represent the parameter of pacing gain. In `PROBE_BW`, BBR uses a set of pacing gain periods $\{1.25, 0.75, 1, 1, 1, 1, 1, 1\}$ while the g_w which represent the cwnd gain is set to 2.

Remark on BBR versions: Figure 1 illustrates the state machine of the original BBRv1 and the above description focuses on this baseline version for analytical clarity. BBRv2 and BBRv3 introduce several refinements (e.g., more conservative probing and tighter loss/ECN handling), but they retain the same model-based control law in (1)–(2) and the core `PROBE_BW` / `PROBE_RTT` cycle. In particular, inflight data is still scaled with $\mathrm{BtlBw} \cdot \mathrm{min_rtt}$ and flows continue to rely on periodic queue draining to refresh the propagation RTT

estimate. As a result, the qualitative mechanisms discussed in this section—how RTT-dependent inflight and probing behavior can lead to persistent queue buildup and unfair bandwidth sharing among heterogeneous RTT flows— also apply to newer BBR versions. Our experimental results in Section V (Figs. 6–8 and 10) confirm that BBRv3, while improved over BBRv1 in some scenarios, still exhibits fairness degradation and queue growth under large buffers and heterogeneous RTTs, consistent with this analysis.

B. Rate Update Mechanism

1) Sliding Window Bandwidth Estimation: Estimated maximum bandwidth ($BtlBw$) in the sliding window, its sample is collected once in each RTT round in all state machines. The maximum value of the last 10 RTTs will be maintained. The update is once per RTT and takes the largest of the last 10 RTT periods.

2) Mandatory Control in `PROBE_RTT`: In `PROBE_BW` state, when BBR determines that the minimum RTT has expired (not updated for more than 10 seconds), it will enter the `PROBE_RTT` mode. In this mode, BBR forces cwnd to be restricted to:

```
tcp_snd_cwnd_set(tp, bbr_cwnd_min_target);
//
cwnd = 4
```

This stage lasts for at least 200 ms, or a complete packet-timed RTT round, to ensure that the bottleneck queue is cleared, thereby accurately re-measuring the propagation delay min_rtt.

C. Convergence Behavior

According to the source code comments and state machine logic of BBRv1:

> "BBR synchronizes flows around the desirable event of an empty bottleneck queue."
> "Large flows entering `PROBE_RTT` drain many packets from the queue, so several flows see a new RTprop (new minimum RTT). This makes their RTprop estimates expire at the same time, so they enter `PROBE_RTT` together, which makes the total queue dip larger and causes more flows to see a new RTprop, and so on. This distributed coordination is the key to both fairness and stability."

This means that BBR flows will complete a synchronous convergence at the moment when they exit `PROBE_RTT` and re-enter `PROBE_BW`. This depends on the fact that all flows will enter `PROBE_RTT` when `min_rtt_stamp + 10s` expires. During the `PROBE_RTT` stage, the cwnd is compressed to the minimum value (4 packets), causing almost no queuing or congestion. During this period, each flow is "draining the queue" and measuring the true `min_rtt`; This mechanism is intended to provide a "fair starting point" for all flows, thereby eliminating bandwidth deviations caused by historical backlogs.

D. Inaction of BBR

To better understand why BBR fails to restore fairness after `PROBE_RTT`, we analyze a simple two-flow scenario in which two long-lived BBR flows share a single bottleneck link without cross traffic. We focus on three representative RTT combinations (both RTTs large, one large and one small, and both small), which cover typical cases observed in practice. For each case, we track how the bottleneck bandwidth (BtlBw) estimate and the inflight data evolve when the flows leave `PROBE_RTT` and re-enter `PROBE_BW`. This analysis shows that, although `PROBE_RTT` is intended to provide a "fair starting point", BBR has no explicit mechanism to correct accumulated queue imbalance, and thus can remain inactive in the face of RTT-driven unfairness.

There are two BBR flows 1 and 2, whose minimum RTT is RTT_1 and RTT_2 respectively. Their $BtlBw$ before they enter the `PROBE_RTT` phase are $BtlBw_1$ and $BtlBw_2$ respectively, and their ratio is $M : N$. There are the following three situations.

1) Case 1: $RTT_1 > 20\,ms$, $RTT_2 > 20\,ms$: At this point, when the `PROBE_RTT` phase (200ms) exceeds 10 RTTs of the two flows, the content of the $BtlBw$ sliding window will be refreshed in `PROBE_RTT`. For each flow, updated maximum BtlBw is equal to

$$BtlBw_{\max} = \frac{4 \cdot \text{BW_UNIT}}{\text{RTT}_m} \quad [\text{pkts}/\mu\text{s}], \text{BW_UNIT} = 2^{24} \tag{3}$$

$$BtlBw_{\max} = \frac{4 \cdot \text{BW_UNIT}}{\text{RTT}_n} \quad [\text{pkts}/\mu\text{s}], \text{BW_UNIT} = 2^{24} \tag{4}$$

when entering the `PROBE_BW` phase. Flow n will take an advantage in the initial state of `PROBE_BW`.

However, flow m has a greater inflight capacity, which allows it to build up more queue occupancy, particularly in networks with deep buffering. This queue dominance leads to a positive feedback loop that favors long RTT flows in BBR. The inflight amount is determined by Equation [2].

So, although both flows begin with the same pacing rate (e.g., $1.25 \times$ BtlBw in the gain cycle), we observe:

- A long RTT flow has a larger inflight budget, allowing it to occupy the bottleneck queue more aggressively;
- In a deep-buffered network, this flow can sustain higher backlog without loss;
- As a result, it receives more ACKs in each RTT and reports a higher delivery rate;
- The estimated $BtlBw$ is updated with this inflated delivery rate, allowing the flow to increase its pacing rate further.

Flow n is gradually suppressed due to the lack of backlog space, the delivery rate remains unchanged, and the $BtlBw$ cannot be updated.

Short RTT flow likes flow n may take an advantage in the beginning in this situation, just like the RTT bias exhibited by traditional loss-based CCAs such as reno [32], but the AIMD mechanism of these CCAs ensures convergence to the Max-Min fair bandwidth allocation [33], [34], [35], while BBR does not enforce any fairness restoration or rate normalization at the

convergence point (i.e., upon exiting PROBE_RTT). The subsequent competition immediately reintroduces and amplifies prior imbalances, especially in deep-buffered networks where long RTT flows can quickly dominate the queue again.

2) Case 2: $RTT_m > 20\,ms$, $RTT_n < 20\,ms$: In this case, at the end of PROBE_RTT, flow n can update its BtlBw as shown by equation [3]. while the BtlBw of flow m still comes from the largest value saved in the previous round of PROBE_BW. After the two re-enter PROBE_BW, the rate ratio becomes:

$$\frac{BtlBw_m}{BtlBw_n} \gg \frac{M}{N} \tag{5}$$

This gap further amplifies the unfairness and BBR does not provide any repair mechanism in its design.

3) Case 3: $RTT_m > 20\,ms$, $RTT_n > 20\,ms$: At this point, neither of them updated the BtlBw during the PROBE_RTT stage. When entering the PROBE_BW stage, they continued to use the old estimated values from the previous round, and the bandwidth allocation remained at the ratio $M : N$, with unfairness remaining. Unless the network environment changes (such as path changes or other flow exits), the issue of fairness will persist for a long time.

E. Insensitivity to Congestion

Without relying on packet loss or delay increase as congestion signals, BBR estimates the network capacity in real time based on $BtlBw$ and min_rtt. However, the attempt to proactively avoid congestion purely through model-driven estimation may be overly optimistic and could fail under certain network dynamics.

BBR's insensitivity to congestion is particularly evident in PROBE_BW, where the pacing rate is periodically increased to search for additional available bandwidth. Despite its exploratory nature, the PROBE_BW phase typically lasts for a fixed duration (10 seconds in BBRv1), regardless of network conditions. During PROBE_BW, when multiple flows coexist, bandwidth overestimation will happen, and there will be continuous queue accumulation or even packet loss. However, BBR cannot respond promptly because it neither interprets the increase in RTT nor packet loss as signals that reduce the transmission rate. This will lead to adverse side effects, such as:

- Excessive queuing delay during bandwidth probing.
- Packet losses are ignored by the algorithm.
- Unfair bandwidth occupation, especially favoring long RTT flows.

In short, BBR's insensitivity to congestion leaves it unaware of emerging network issues such as fairness degradation, and prevents it from taking corrective actions.

IV. BBR With Extended State (BBR-ES)

A. Overview of BBR-ES

To fundamentally address the inaction of traditional BBR at the convergence point and the unfair bandwidth competition in the presence of RTT heterogeneity, BBR-ES introduces two key enhancements, directly motivated by our analysis in Section III:

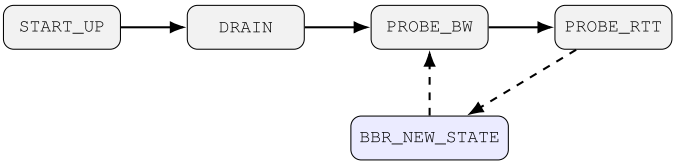

Fig. 2. State machine of BBR-ES. Gray nodes denote the original BBR states (START_UP, DRAIN, PROBE_BW, PROBE_RTT), and the blue node denotes the new stabilization state (BBR_NEW_STATE). Solid arrows represent the standard transitions inherited from BBR, while dashed arrows represent the additional trend-driven transitions introduced by BBR-ES.

1) **Extended State** (BBR_NEW_STATE): a newly designed state added to the original BBR state machine. Traditional BBR exhibits *inaction* after convergence—upon exiting PROBE_RTT, it directly re-enters PROBE_BW with no mechanism to resolve pre-existing unfairness or queue buildup. To address this, we introduce BBR_NEW_STATE as a post-convergence stabilization phase. It gradually ramps the sending rate based on recent RTT trends and time-in-state, enabling fairer and smoother recovery, particularly for disadvantaged flows.
2) **Trend-Driven Mode Transition Mechanism**: a dynamic mode switching mechanism that leverages short-term trends in bottleneck bandwidth ($BtlBw$) and round-trip time (RTT). Unlike prior delay-based methods that impose global rate reduction on all flows—a *one-size-fits-all* approach that penalizes low-impact flows equally—our mechanism selectively triggers PROBE_RTT only for flows that exhibit behaviors consistent with excessive queue contribution. This strategy enables BBR-ES to be more aware of the congestion and queue in the buffer, and to enter fair convergence more frequently and precisely, working synergistically with BBR_NEW_STATE to ensure fairness is restored promptly.

Figure 2 summarizes the resulting state machine. The four original BBR states (START_UP, DRAIN, PROBE_BW, and PROBE_RTT) form the standard pipe-filling and steady probing cycle, while the new BBR_NEW_STATE acts as a short stabilization phase between PROBE_RTT and PROBE_BW. Solid arrows denote the transitions inherited from BBR, and dashed arrows denote the additional trend-driven transitions that allow BBR-ES to re-enter the stabilization state when persistent congestion patterns are detected.

B. BBR_NEW_STATE: Stabilization Phase

The BBR_NEW_STATE is designed to absorb queue-related feedback after convergence and enable fairer recovery into the bandwidth probing cycle. Unlike BBR, which resumes full pacing gain immediately after PROBE_RTT, BBR-ES uses a multi-stage gain schedule:

This scheduling is time-driven and state-aware. It is designed with the following rationale:

- The initial gain (1.5) allows the flow to re-establish a basic sending rate after exiting PROBE_RTT, enabling quick resumption of throughput without being overly aggressive.

```
while (mode == BBR_NEW_STATE) {
time_in_state = now -state_enter_time;
if (time_in_state <= 50000){
pacing_gain = 1.5;
}
else if (time_in_state <=100000) {
pacing_gain = (srtt +min_rtt) / (2 *
min_rtt);
}
else {
pacing_gain = (2.0 *min_rtt) / srtt;
} cwnd_gain = pacing_gain;
}
```

- The intermediate phase applies an RTT-aware pacing gain. This value moderately increases with queue buildup but remains flow-local and self-adaptive. Unlike delay-based mechanisms that uniformly suppress all flows, this design allows each flow to gradually probe bandwidth according to its own queue status, enabling a fair and smooth recovery process. This phase explicitly accepts short-term queue buildup as a natural part of convergence and avoids premature or excessive rate suppression. Since the pacing gain is capped and inflight is regulated by RTT, this stage does not introduce significant delay expansion or instability.
- The final phase adopts a reverse-ratio function $(2 \cdot \texttt{min_rtt})/\texttt{srtt}$ to penalize persistently elevated RTT. It serves as a corrective measure to suppress flows that become overly aggressive during recovery. By gradually reducing the sending rate of these flows, BBR-ES restores long-term fairness while preventing queue dominance.

Here, `time_in_state` is measured in microseconds since the flow enters `BBR_NEW_STATE`, so the two thresholds 50,000 and 100,000 correspond to short durations of roughly 50 ms and 100 ms. They split `BBR_NEW_STATE` into three brief sub-intervals: an initial phase to leave the very low rate of `PROBE_RTT`, an intermediate phase to hold an RTT-aware gain long enough to observe whether RTT keeps increasing, and a final phase to drain excess inflight if congestion persists. These values were chosen based on empirical tuning across our Mininet and EC2 experiments covering RTTs from tens to hundreds of milliseconds and different bottleneck settings. We observed that significantly smaller thresholds make `BBR_NEW_STATE` too short to correct unfairness, whereas much larger thresholds only slow down convergence and slightly increase queuing delay. The qualitative behavior of BBR-ES is robust across a reasonable range of such thresholds, so they should be viewed as practical engineering defaults rather than fragile, finely tuned parameters.

C. Trend-Driven Transition Logic

In original BBR, `PROBE_RTT` is triggered only when the min_rtt estimate expires (every 10 seconds). This reactive approach fails to address persistent queue buildup or fairness degradation in time.

BBR-ES introduces a **trend-based trigger**, which proactively transitions to `PROBE_RTT` if short-term indicators suggest a flow is inflating the queue. The pseudo code logic is:

```
record_history(BtlBw, RTT); // every 100
ms
if (increasing_trend(BtlBw) &&
increas-ing_trend(RTT)){
enter_PROBE_RTT();
}
```

We define an increasing trend as a majority of 8 sliding-window samples (e.g., 5/8) showing upward movement. The rationale is:

- **Case 1**: If `BtlBw` is decreasing and `RTT` decreasing as well, we let the flow probe the available bandwidth.
- **Case 2**: If `BtlBw` is increasing but `RTT` remains stable or decreases, it implies that capacity is being released (e.g., a flow exits or a queue drains). In this case, there is no need to reduce the rate.
- **Case 3**: If both `BtlBw` and `RTT` increase, it indicates excessive sending or queue competition. RTT elevation specifically signals that more packets are staying in the queue longer—a sign of congestion and unfair probing. Early entry into `PROBE_RTT` is then warranted to drain the queue and reset fairness.

Why fixed sampling interval?:

Rather than sampling every RTT or every probing cycle, BBR-ES uses a **fixed interval of 100 ms** for trend recording. This choice ensures fairness across flows with heterogeneous RTTs. If trend detection were bound to RTT cycles, short RTT flows would be sampled and evaluated more frequently, making them more likely to enter `PROBE_RTT` prematurely. This would ironically amplify the unfairness. The fixed interval ensures consistent observation granularity and synchronized decision-making regardless of RTT disparity.

Design Scope: We intentionally keep the `START_UP` phase identical to the original BBR. BBR-ES targets the post-convergence regime where flows compete for long periods and RTT unfairness becomes pronounced. In this regime, queue buildup and bandwidth imbalance mainly arise during repeated *PROBE_BW*/*PROBE_RTT* cycles, not while the pipe is still being filled. Modifying `START_UP` to use the same extended-state logic would slow down pipe filling and penalize short-lived flows, without providing additional RTT fairness benefits. Therefore, the BBR_NEW_STATE and trend-based trigger are only activated after the flow has entered the steady probing regime.

This selective response strategy avoids the global overreaction of delay-based methods, where even lightly contributing flows suffer rate reduction. BBR-ES enables differentiated control based on actual trend behavior, improving fairness and delay without sacrificing bandwidth efficiency.

D. Convergence Properties of BBR-ES

Although BBR-ES extends the original BBR state machine with an additional intermediate state and a trend-driven PROBE_RTT trigger, its behavior remains that of a finite-state, model-based congestion control algorithm with bounded transitions.

First, BBR-ES operates over a finite set of modes (`START_UP`, `DRAIN`, `PROBE_BW`, `PROBE_RTT`, and `BBR_NEW_STATE`), and all state transitions are explicitly guarded by time windows and sliding-window measurements of bottleneck bandwidth and RTT. The trend detector in `bbr_update_min_rtt()` records the per-flow bottleneck bandwidth and RTT samples into fixed-size histories of length $N = 8$ with a sampling interval of 100 ms. It triggers an early PROBE_RTT only when *both* bandwidth and RTT exhibit a majority-increasing trend in this finite window. After triggering, the history is reset and a new window is started. Therefore, the trend-based trigger cannot oscillate arbitrarily fast or accumulate unbounded state.

Second, the additional BBR_NEW_STATE is explicitly time-bounded. When leaving PROBE_RTT, BBR-ES enters BBR_NEW_STATE and records the entry time. Within this state, the pacing and congestion window gains are gradually adjusted in three phases: (i) an initial phase with a modest gain (e.g., 1.5) to recover smoothly from the low rate in PROBE_RTT; (ii) an intermediate phase where the gain is adapted using the ratio between the smoothed RTT and the minimum RTT; and (iii) a final phase where the gain approaches a stable value as the measured RTT converges toward the propagation RTT. Importantly, the dwell time in BBR_NEW_STATE is upper-bounded by $\max(100\text{ms}, 3 \cdot \text{min_rtt})$, after which the algorithm deterministically returns to PROBE_BW. Thus BBR_NEW_STATE cannot persist indefinitely and cannot introduce unbounded oscillations.

Finally, congestion window updates in BBR_NEW_STATE follow a diminishing-increment rule, where the increment decreases as the cwnd grows, and are still capped by the model-based BDP target in `bbr_set_cwnd()`. Flows that contribute more heavily to persistent queue buildup are pushed into `PROBE_RTT` and BBR_NEW_STATE more frequently, while flows with smaller inflight and weak queue-inflating trends can remain in PROBE_BW for longer periods. As a result, BBR-ES converges to a steady-state operating regime in which flows cycle between `PROBE_BW` and PROBE_RTT, with BBR_NEW_STATE acting as a short, bounded stabilization phase after each PROBE_RTT episode; the bottleneck queue stays bounded and bandwidth sharing becomes more balanced across RTT-heterogeneous flows. Our long-duration experiments in Sections V and VI empirically confirm this behavior: after the initial adaptation, BBR-ES maintains stable bandwidth sharing, bounded queuing delay, and high link utilization under fixed network conditions.

Design Rationale and Optimization Scope:

BBR-ES is designed as a practical enhancement to the BBR framework rather than as a mathematically optimized solution. This choice is deliberate. Modern Internet congestion control operates under highly dynamic and partially observable conditions—where path asymmetry, bursty cross-traffic, and fluctuating RTTs prevent closed-form optimization models from accurately capturing real behaviors. In such environments, optimization-based schemes (e.g., those requiring convexity, full-queue observability, or precise propagation-delay estimation) are not deployable in real systems. Our goal is therefore to develop a lightweight, deployable mechanism guided by measurable trends—such as inflight evolution, bandwidth variation, and RTT dynamics—while remaining fully compatible with the BBR model.

Computational Complexity: BBR-ES preserves the per-flow, per-ACK update structure of the original BBR implementation. On each control update, BBR-ES performs a constant amount of additional work: it updates a small, fixed-size sliding window of recent bandwidth, RTT, and inflight samples (e.g., 8 entries) and computes simple aggregates (minimum, maximum, and finite differences) over this window. There are no operations whose cost grows with the number of flows or the path bandwidth–delay product. Therefore, the per-ACK computational complexity of BBR-ES remains $O(1)$, and the extra per-flow memory footprint is also $O(1)$, consisting of a few scalar variables and a handful of history samples. In practice, the CPU overhead is comparable to that of BBR, as confirmed by our Mininet and EC2 experiments, and BBR-ES does not introduce any centralized or cross-flow coordination.

Although BBR-ES is heuristic by design, it follows clear principles: (i) ensuring that inflight converges toward the fair-share BDP; (ii) detecting persistent congestion through multi-RTT trend signals rather than instantaneous noise; and (iii) avoiding unnecessary rate cuts for flows that contribute minimally to the bottleneck queue. Our evaluation demonstrates that such a principled heuristic design yields strong convergence behavior and fairness improvements across both controlled testbeds and real-world EC2 networks.

Importantly, BBR-ES does not aim to increase the steady-state sending rate beyond that of existing BBR variants. In the PROBE_BW state, it reuses the same pacing_gain and cwnd_gain configuration as BBRv1/v3. The additional BBR_NEW_STATE and the trend-driven transitions mainly change *how often* a flow temporarily leaves PROBE_BW to refresh its operating point, rather than *how fast* it probes in steady state. By periodically re-entering a draining phase when persistent queue buildup or unfairness is detected, BBR-ES can flush excess buffer occupancy and restore RTT fairness while keeping throughput high. Because the steady-state gains in PROBE_BW are unchanged, BBR-ES is not more aggressive than BBRv1/v3 when competing with loss-based flows such as CUBIC; instead, it tends to behave more gently by avoiding long periods of sustained queuing.

We note that, in principle, part of the functionality of `BBR_NEW_STATE` could be embedded into the existing `PROBE_BW` and `PROBE_RTT` modes by modifying their gain schedules and transition conditions. In BBR-ES we instead choose to make the post-`PROBE_RTT` recovery an explicit intermediate state for two reasons. First, it allows us to keep the original `PROBE_BW` structure and steady-state gain pattern unchanged, which simplifies reasoning about

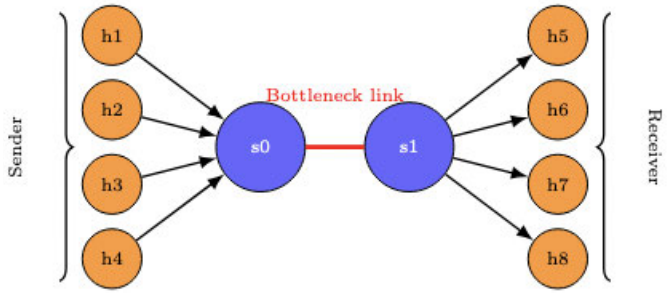

Fig. 3. Mininet topology.

backward compatibility with existing BBR deployments. Second, concentrating the RTT-aware ramp-up and fairness-restoration logic in a short-lived state with bounded dwell time makes the behavior easier to analyze, tune, or disable, while the implementation overhead in the Linux TCP stack remains small (a modest extension of the existing BBR module without additional kernel dependencies).

In summary, the `BBR_NEW_STATE` and trend-driven transition logic complement each other: the former restores fairness post-convergence, and the latter ensures convergence occurs at the right time. Together, they enable BBR-ES to more fairly and responsively manage congestion across diverse network conditions.

V. Evaluations With Controlled Networks

In this section, we validate the performance of our BBR-ES through the experimental evaluations via Mininet.

A. Evaluation Settings

The sizes of data packets and ACK packets are 1,500 bytes and 40 bytes, respectively. Each experiment runs for 200 seconds. The experiments are repeated 10 times, and the results are averaged across the 10 trials. Sender is based on Linux kernel version 6.4.0, and iPerf2 is used to send packets from the sender.

Each sender is equipped with the following congestion control algorithms: BBR-ES, BBRv1 [1], BBRv3 [16], BBR-Revised (BBR-R) [25], CUBIC [18], DA-BBR [11]. These algorithms are implemented based on their Linux source code.

We assess the throughput fairness, the queuing delay, and the bottleneck link utilization of flows. We use Jain's fairness index [36] to measure the fairness. Our controlled experiments therefore focus on BBR-family flows sharing a single bottleneck and compare BBR-ES against BBRv1, BBRv3, BBR-R, CUBIC, and DA-BBR. CUBIC is included as a representative loss-based baseline, but we do not attempt to fully characterize cross-protocol bandwidth sharing; instead, we ensure that BBR-ES does not exhibit more aggressive behavior than existing BBR variants in terms of utilization and queuing delay.

B. Mininet Topology

Figure 3 illustrates the network topology used in Mininet. We set up four nodes $(h1, h2, h3, h4)$ as the sender, two nodes $(s0, s1)$ as the switch, and four nodes $(h5, h6, h7, h8)$ as the receiver. Nodes used as the sender are all connected via independent access links to the bottleneck, which is composed of $s0$ and $s1$. $s1$ is connected to the receiving nodes. Access links have a bandwidth of 1000 Mbps, while the bottleneck bandwidth is set to 100 Mbps with a 5 ms one-way propagation delay.

To ensure diversity in RTT conditions, we use three representative combinations of round-trip delays without queuing at the bottleneck link: (14 ms, 186 ms), (30 ms, 170 ms), and (90 ms, 110 ms). For each flow count n, the flows are evenly split between the two RTT values. For example, in the 2-flow case, one flow has 14 ms RTT and the other has 186 ms; for 4 flows, two are assigned 14 ms and two are assigned 186 ms; for 8 flows, four are assigned 14 ms and the other four are assigned 186 ms, and so on. This setup allows us to emulate asymmetric competition scenarios where flows with significantly different RTTs coexist and contend for the bottleneck link. Based on this topology and RTT configuration, we next examine how BBR-ES and the baseline algorithms behave under two key dimensions: (i) the number of concurrent flows sharing the bottleneck, and (ii) the buffer provisioning at the bottleneck link. Subsection V-C varies the number of flows under a fixed buffer configuration, while Subsection V-D varies the buffer size to study its impact on fairness, utilization, and queuing delay. These values are chosen to cover both highly heterogeneous and near-homogeneous RTT regimes that we target in this work. The 5 ms one-way propagation delay on the bottleneck link yields a minimum RTT on the order of 10–20 ms, which is typical for well-provisioned metro or intra-region paths; using much smaller base RTTs would be less realistic for the wide-area scenarios we aim to model. The pair (14 ms, 186 ms) represents an extreme short-vs-long RTT case similar to an access flow competing with a remote-region flow. The pair (30 ms, 170 ms) corresponds to a moderately asymmetric wide-area path, while (90 ms, 110 ms) captures a near-symmetric case where RTTs are close. Together, these three combinations allow us to stress-test BBR-ES under both severe RTT disparity and mild RTT differences within a single unified setup.

C. Effect of Number of Flows

We first study how the number of coexisting flows affects fairness, link utilization, and queuing delay under the Mininet topology described above, using a fixed bottleneck buffer configuration. We only use $h1$ and $h2$ as the sender in the case of two flows. We use concurrent flows when the number of flows is beyond four.

Figure 4 shows the throughput and link utilization under the effect of the number of flows and three representative RTT combinations. Each point represents an experiment with a specific number of flows (indicated by the point label). Across all three scenarios, BBR-ES consistently occupies the top-right region of the plots, representing near-optimal fairness (Jain's index 0.95) and high link utilization (above 98% in most cases). This visually highlights its ability to balance fairness and efficiency across heterogeneous RTT environments, even as the number of flows increases. In contrast, BBRv1

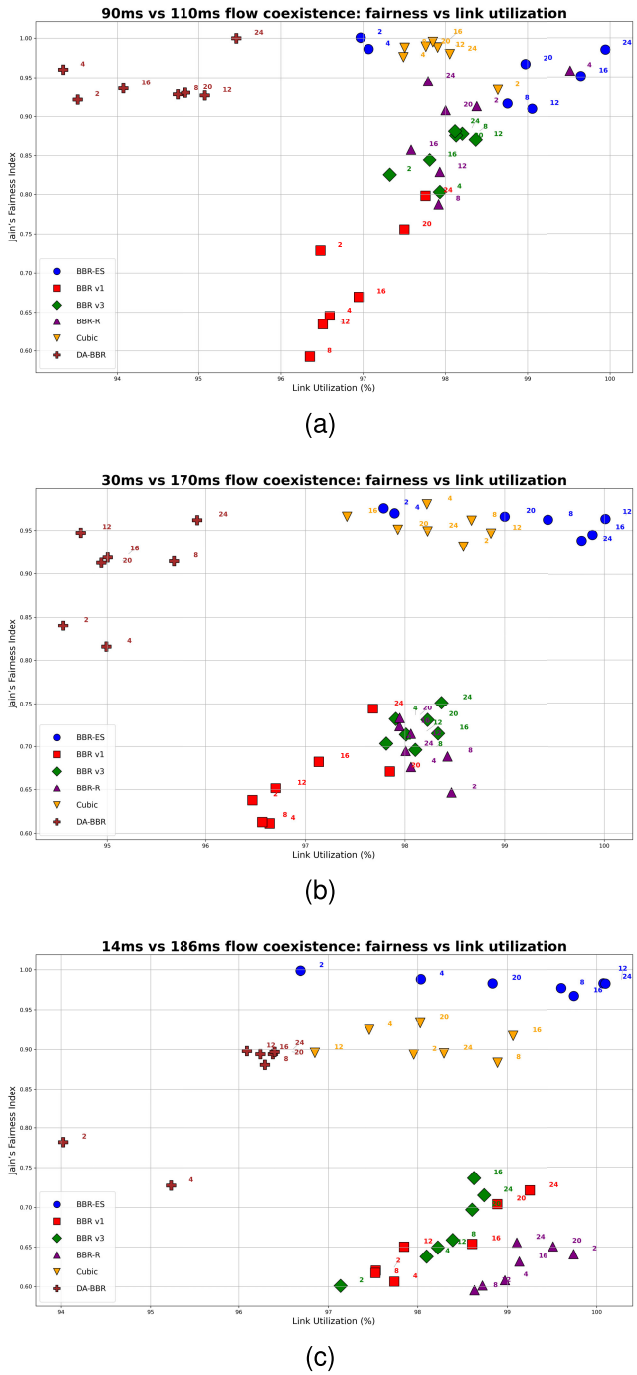

Fig. 4. Effect of number of flows under different RTT combinations.

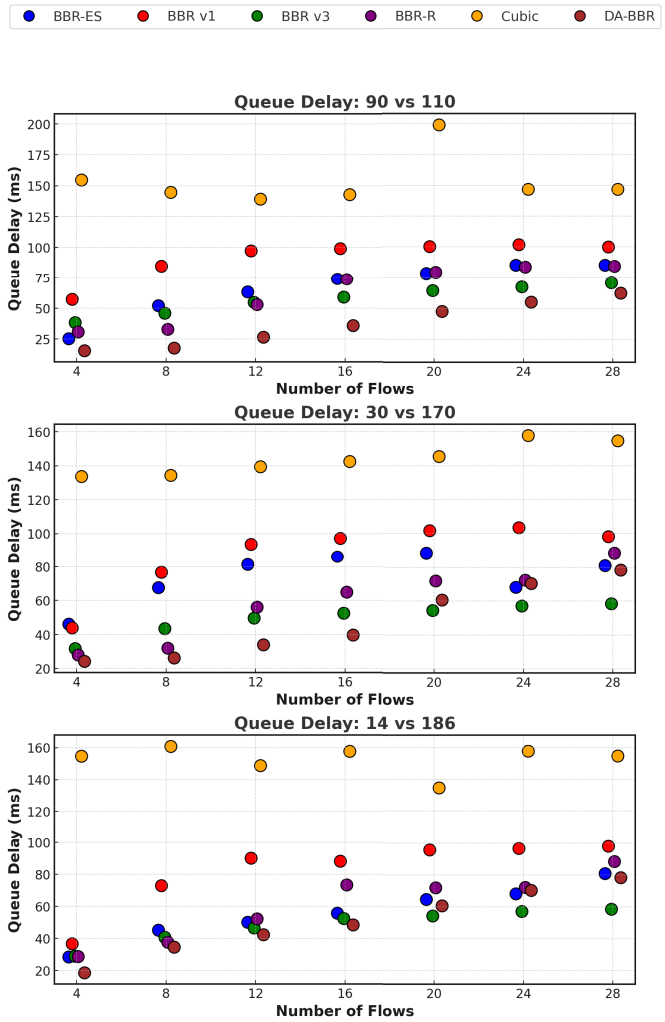

Fig. 5. Effect of number of flows on queuing delay.

and BBRv3 maintain high utilization, but fairness degrades significantly with larger RTT disparity or higher flow counts, often falling below a Jain index of 0.7. CUBIC performs relatively well in moderate RTT scenarios. BBR-R provides slightly better utilization than BBRv1 and BBRv3 but suffers from noticeable fairness imbalance. DA-BBR achieves high fairness but frequently sacrifices utilization.

Figure 5 shows the average queuing delay across different numbers of flows under the same RTT combinations used in Figure 4. As expected, queuing delay increases as more concurrent flows share the bottleneck. Among all algorithms, CUBIC exhibits the highest queuing delay across all scenarios, often exceeding 140 ms. BBRv1 and BBRv3 also accumulate substantial queue buildup as the number of flows increases.

In contrast, DA-BBR achieves the lowest delay in several settings due to its strong delay-sensitive control, but this comes at the cost of reduced throughput or fairness in other cases (see Figure 4). BBR-ES does not always achieve the minimum delay, but it consistently maintains moderate queuing across all flow counts and RTT combinations. Its queuing delay remains stable and clearly bounded, and is substantially lower than that of BBRv1, BBRv3, and CUBIC. In this sense, BBR-ES is designed to provide a balanced delay–fairness–utilization trade-off rather than aggressively minimizing delay alone.

Overall, BBR-ES provides a balanced design: it controls delay effectively without over-constraining the sending rate, achieving a favorable balance between fairness, utilization, and latency under the effect of the number of flows.

D. Effect of Buffer Size

Building on the above results for varying flow counts, we next examine how the bottleneck buffer size itself influences fairness, utilization, and queuing delay. Figure 6 illustrates the

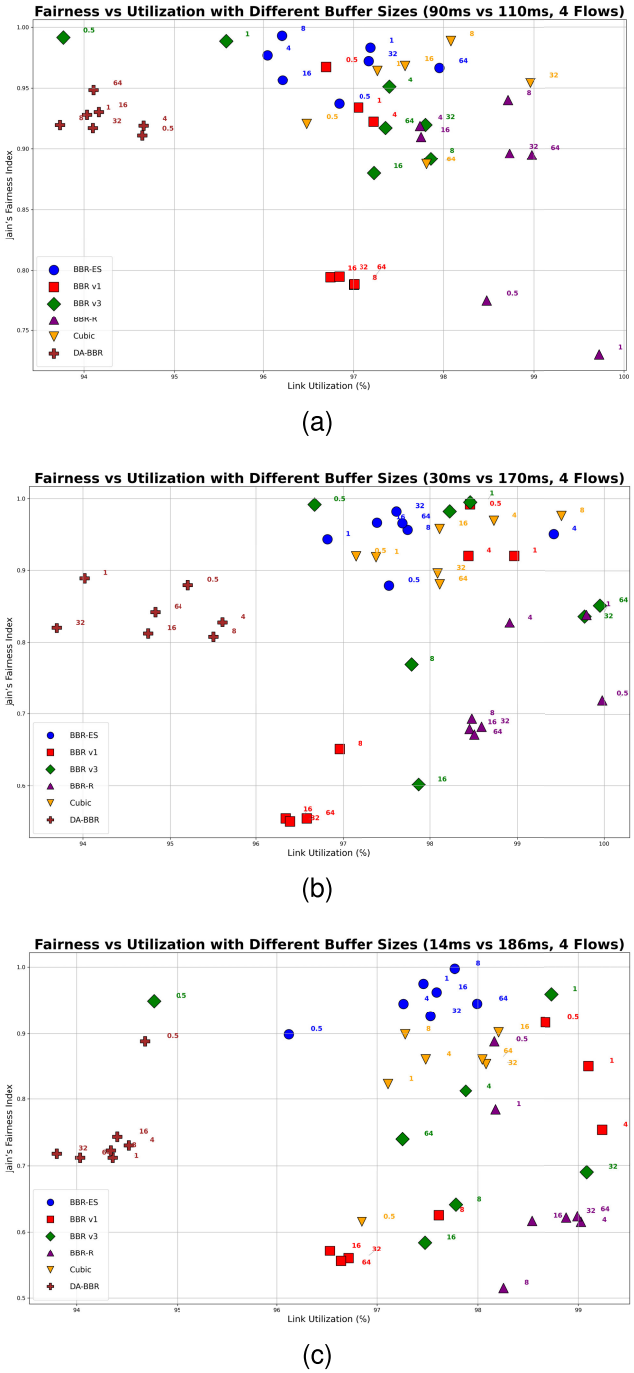

Fig. 6. Effect of buffer size under different RTT combinations (4 flows).

impact of buffer size on fairness and link utilization under different RTT combinations with four coexisting flows. Each point represents an experiment with a specific buffer size (in units of BDP, indicated by the point label). We vary the buffer from 0.5 BDP to 64 BDP in order to cover both shallow-buffer settings (e.g., AQM-style or data-center-like configurations) and deep-buffer settings commonly found in commodity routers. This range allows us to observe how sensitive each algorithm is to under- and over-provisioned queues.

Across all three RTT settings, BBR-ES shows stable performance and almost always remains in the top-right region of each plot, achieving both high fairness and high utilization over the entire buffer range. This robustness indicates that BBR-ES effectively adapts to varying buffer conditions without sacrificing fairness or efficiency.

In contrast, BBRv1 and BBRv3 display strong sensitivity to buffer size, with fairness fluctuating significantly. BBRv1 often suffers from low fairness when the buffer is either too small or too large, and BBRv3 shows marked degradation as the buffer increases. BBR-R maintains relatively high utilization but performs poorly in fairness, particularly with larger buffers. CUBIC and DA-BBR achieve reasonable fairness at certain buffer sizes, but their behavior is less consistent across scenarios.

Figure 7 shows the results of 12 flows. Compared to the 4-flow case, the increase in flow count introduces greater competition and congestion, which amplifies the differences between congestion control algorithms. BBR-ES continues to demonstrate robust performance across all buffer sizes. BBRv1 and BBRv3 exhibit clear signs of degradation as the buffer increases, with fairness dropping below 0.7 in many cases. CUBIC and DA-BBR occasionally achieve high fairness, but often suffer from reduced utilization or inconsistent performance across buffer sizes. BBR-R tends to maintain good utilization, but at the cost of fairness, especially in large buffer settings.

Figure 8 shows the queuing delay as a function of buffer size under different RTT combinations, for both 4-flow and 12-flow scenarios. In general, queuing delay increases with larger buffers and more concurrent flows, as expected.

CUBIC exhibits the most pronounced queue buildup, with delay exceeding 500 ms at large buffer sizes. This highlights its aggressive probing and lack of delay awareness. BBRv1 and BBRv3 also show significant queue growth, particularly with 12 flows and large buffers. DA-BBR maintains low delay overall, especially under small to medium buffer sizes, but as shown in previous figures, it often compromises utilization and fairness to do so.

BBR-ES maintains bounded and stable queuing delay across all tested buffer sizes and round-trip time (RTT) settings. BBR-ES consistently achieves very low delay in most scenarios, often within the top-performing group. In some settings, it closely approaches or matches the delay levels of DA-BBR, while maintaining high fairness and throughput simultaneously. This reinforces BBR-ES's design goal: to control delay without suppressing throughput or fairness, even under increasing congestion and buffer pressure.

Remark on Bottleneck Bandwidth: In the Mininet experiments, we set the bottleneck link capacity to 100 Mbps while keeping the access links at 1 Gbps. This setting is sufficient to evaluate the behavior of BBR-ES for two reasons. First, 100 Mbps is representative of many access and intermediate links in today's networks, and it already creates non-trivial contention among multiple long-lived flows under the RTT and buffer configurations in Section V. Second, BBR-family algorithms are primarily governed by the bandwidth–delay product (BDP) rather than the absolute link rate. Given a fixed BDP,

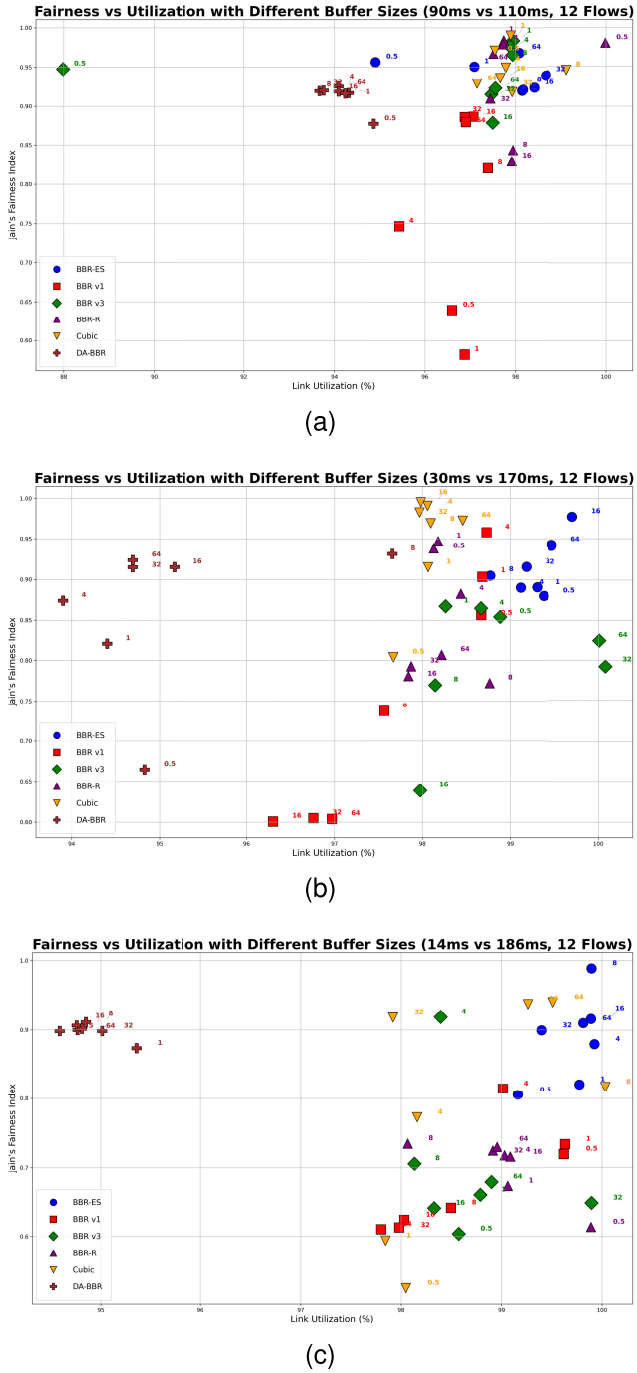

Fig. 7. Effect of buffer size under different RTT combinations (12 flows).

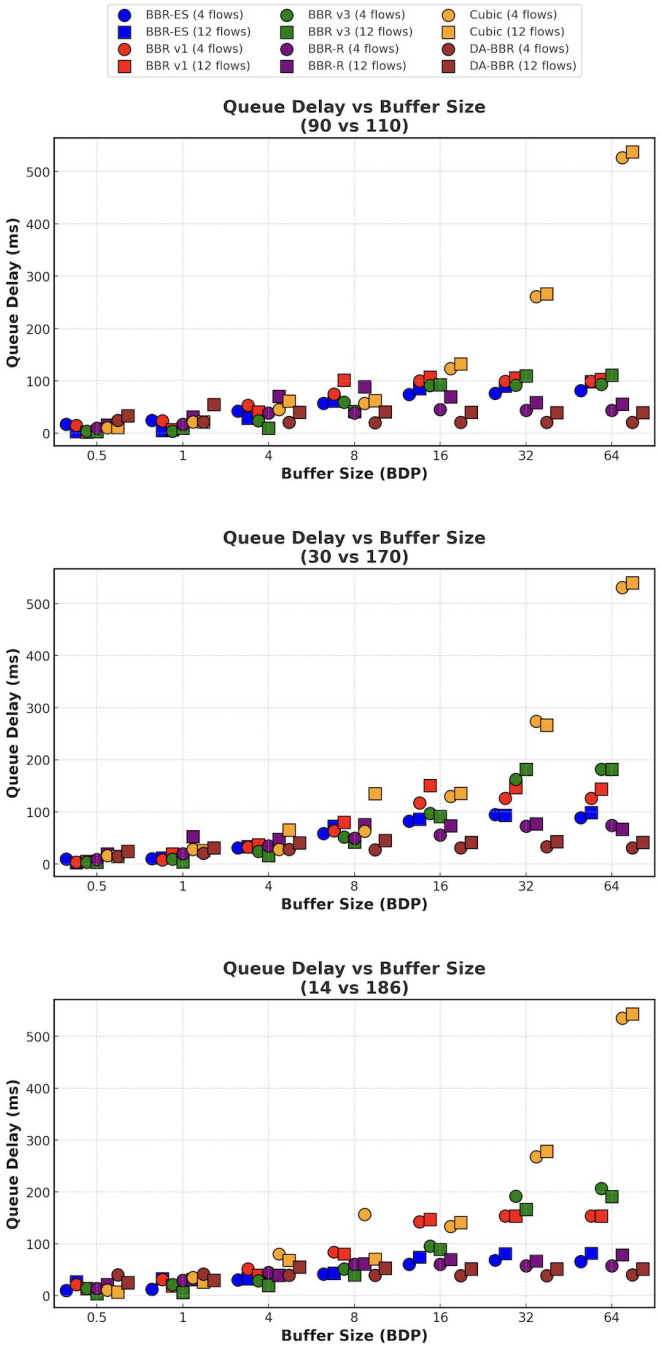

Fig. 8. Effect of buffer size on queuing delay.

increasing the raw bandwidth while scaling RTT accordingly does not change the qualitative model-based dynamics of BBR or BBR-ES; the same fairness and queuing phenomena appear at higher rates.

Our Mininet-based experiments systematically evaluated the effect of both the number of flows and buffer size across three representative RTT combinations. We compared BBR-ES with multiple congestion control algorithms. The results demonstrate that BBR-ES consistently achieves high fairness and link utilization while maintaining low queuing delay across diverse scenarios. Compared to other algorithms, BBR-ES offers the best overall balance across all performance metrics, showcasing strong robustness in heterogeneous network conditions.

VI. Evaluations With Actual Internet

A. EC2-Based Topology

While the Mininet-based experiments in Section V allow us to precisely control RTTs, buffer sizes, and flow counts, they cannot fully capture the path asymmetry, background traffic, and temporal variability that arise on the real Internet. To assess whether the advantages of BBR-ES carry over to

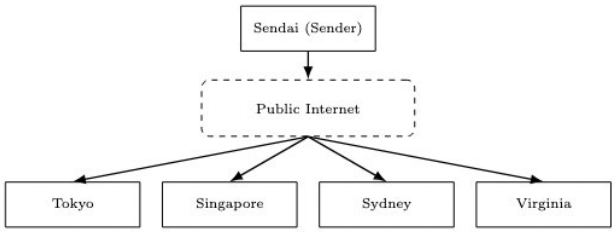

Fig. 9. EC2-based topology.

practical deployments, we complement the controlled experiments with measurements over actual wide-area paths. In this section, we use Amazon EC2 instances deployed across multiple geographic regions to evaluate BBR-ES under real-world routing, cross traffic, and RTT fluctuations, and compare its behavior against BBRv1, BBRv3, CUBIC, and other BBR variants.

We set up a server in Sendai (Japan) as the sender and four receivers in Tokyo (Japan), Singapore, Sydney (Australia), and Virginia (America) as shown by Figure 9. Unlike the emulated testbed in Mininet, the EC2 network paths are uncontrollable and represent real Internet routes with dynamic congestion, asymmetric routing, and heterogeneous RTTs. This setup effectively forms an opaque wide-area network, allowing us to test BBR-ES under diverse and unpredictable conditions.

We use the same basic experimental methodology as in Section V-A: data packets of 1,500 bytes and 40 byte ACKs, long-lived iPerf2 flows running for 200 s, and each configuration repeated 10 times with results averaged over the trials. We again evaluate throughput, Jain's fairness index, queuing delay, and link utilization. The main difference from the Mininet setup is that, on EC2, we cannot control the internal network path; instead, we only configure the server's Ethernet speed (100, 200, or 300 Mbps) by disabling auto-negotiation on the NIC, while all other parameters follow the settings in Section V-A.

1) Effect of Number of Flows Under Different Bottleneck Bandwidth: Figure 10 presents the relationship between link utilization and fairness as the number of concurrent flows increases, under three bottleneck capacities: 100 Mbps, 200 Mbps, and 300 Mbps. Each point represents an experiment with a specific number of flows (indicated by the point label). we consider six configurations with 4, 8, 12, 16, 20, and 24 concurrent long-lived flows; for each configuration, all flows are started simultaneously and remain active for the entire measurement interval.

Across all bandwidth settings, BBR-ES consistently appears in the upper-right region of each plot, indicating its ability to maintain both high fairness and high utilization across varying flow intensities. This is particularly evident at 300 Mbps, where BBR-ES continues to outperform others even under higher throughput demands.

BBRv1 shows consistently poor fairness, especially as the number of flows increases. BBRv3 and BBR-R occasionally achieve good fairness, but their performance is less stable, with more variance across flow counts and bandwidths. CUBIC

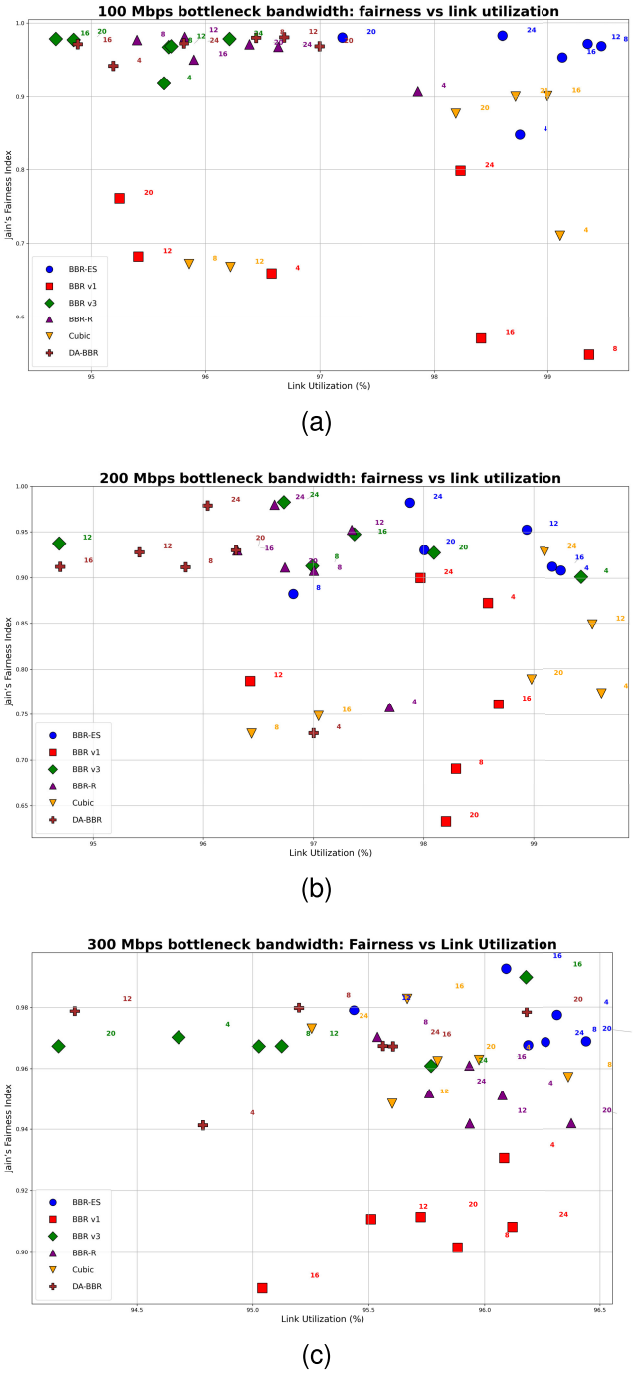

Fig. 10. Effect of number of flows based on EC2.

exhibits moderate fairness but suffers from low utilization at lower flow numbers. DA-BBR performs well in fairness in some scenarios but lacks consistent efficiency, and its link utilization is lower.

Figure 11 shows the RTT variation across four EC2 receiver locations—Tokyo, Singapore, Virginia, and Sydney, under different flow counts and bottleneck bandwidths. These results reflect real Internet environments with long propagation delays and diverse path conditions. Overall, with the increase in the number of Flows, the RTT of each algorithm rises significantly under high-bandwidth conditions, especially CUBIC

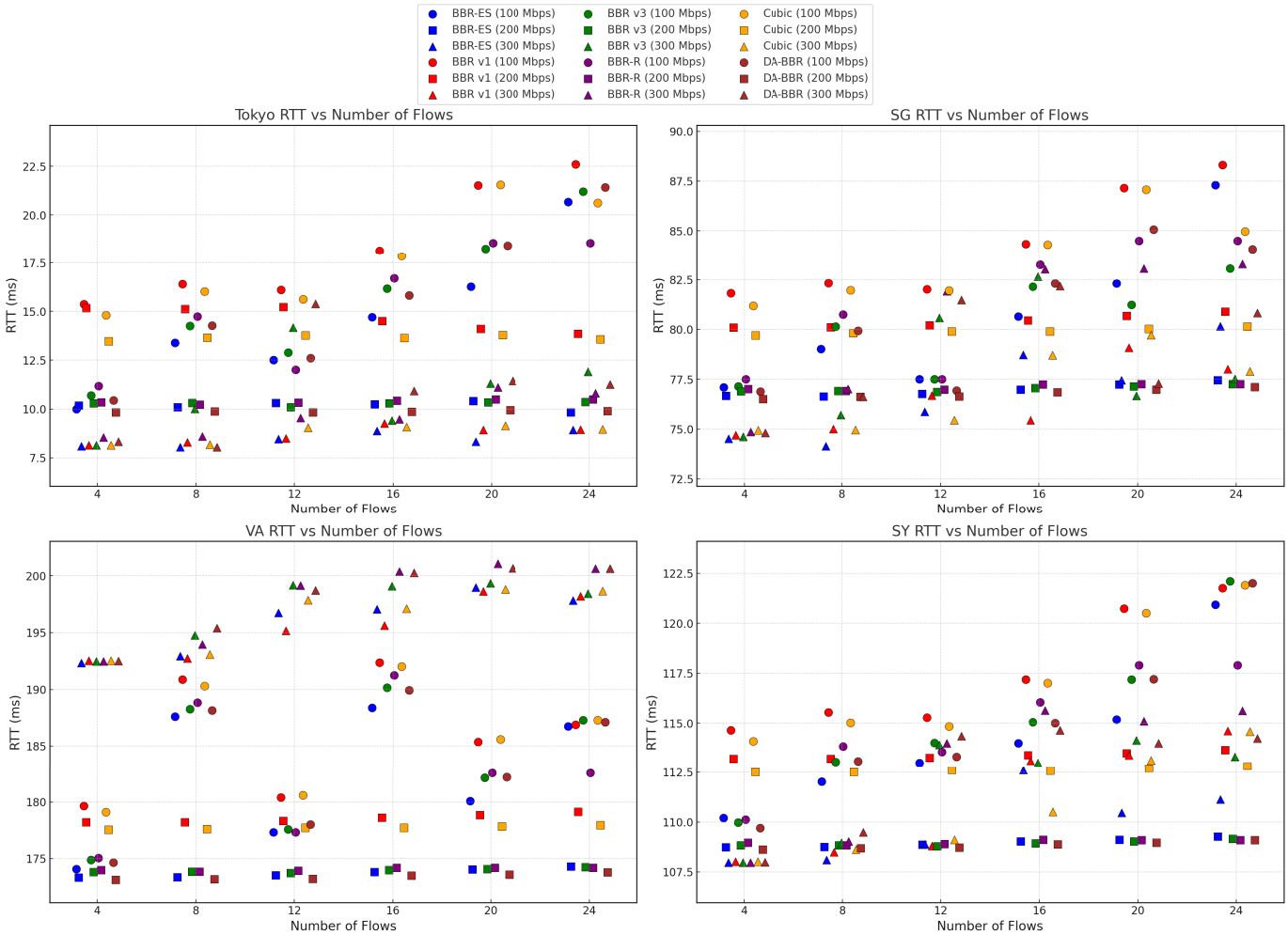

Fig. 11. Effect of number of flows on RTT.

and BBRv1, which accumulate significant queuing delays on paths with lower delays, such as Tokyo and Singapore. BBR-R also shows an increase in RTT when the number of flows is large, which may be related to its more aggressive detection behavior and the lack of fine delay control. Under all locations and network Settings, the RTT of BBR-ES is almost the lowest among all algorithms. Even if the flow number increases to 24, it can always maintain a low RTT.

These results confirm that BBR-ES maintains its strong performance not only in controlled environments like Mininet but also in actual network environments.

It adapts effectively to both flow scale and available bandwidth while sustaining a balanced trade-off between throughput and fairness.

2) Discussion: Why Ec2 Results Differ From Mininet:

- **Lack of control and observability.** In EC2, the bottleneck may exist at any point along the transmission path, including NATs, routers, or backbone infrastructure. These are neither visible nor configurable, in contrast to the fully controlled Mininet setup.
- **Asymmetric ACK return paths.** Flows directed to geographically distant nodes (e.g., Tokyo and Singapore) often traverse different physical paths. Unequal reverse-path latency affects ACK timing, causing BBR-based algorithms to misinterpret bandwidth availability.
- **RTT measurement distortion due to jitter.** BBR and its variants depend on accurate RTT measurements to estimate minimum RTT (`RTprop`). In real-world conditions, jitter introduces noise, undermining RTprop estimation and triggering incorrect mode transitions.
- **Hidden queuing and shaping mechanisms.** Intermediate devices may implement traffic shaping, ECN, or AQM schemes that affect packet behavior but remain opaque to the sender, further complicating congestion inference.
- **Cross-region characteristics.** Trans-Pacific transmissions involve more complex routing, higher baseline RTT, and a larger number of intermediate nodes, amplifying the unpredictability of network behavior.

While the real-network experiment lacks the controllability of emulation (e.g., unobservable bottleneck location, dynamic reverse-path jitter, and unknown queuing strategy), it faithfully captures deployment-level uncertainty. As shown in Fig. 10, across three bottleneck capacities (100, 200, and 300 Mbps) and up to 24 concurrent flows, BBR-ES consistently remains in the upper-right region of the utilization–fairness plane, whereas CUBIC, BBRv1, and BBR-R move toward lower

TABLE I
AVERAGE PER-CORE CPU UTILIZATION ON SENDER HOST (24 CONCURRENT FLOWS)

Algorithm	Avg. CPU utilization (%)
BBR v3	0.27
BBR v1	0.34
BBR-R	0.31
DA-BBR	0.37
BBR-ES	0.35
CUBIC	0.50

fairness or lower utilization as capacity and flow count increase. Fig. 11 further shows that for all four EC2 regions (Tokyo, Singapore, Sydney, and Virginia), the average RTT of BBR-ES stays close to the baseline RTT even under heavy load, while other algorithms exhibit pronounced RTT inflation. Therefore, despite the inherent variability of the wide-area Internet, BBR-ES provides consistently high utilization, high fairness, and low delay across diverse paths, link capacities, and flow scales, which is precisely what we mean by robustness and scalability in real-world scenarios.

In summary, the EC2-based evaluations confirm that the trends observed in our controlled Mininet testbed carry over to real Internet paths: BBR-ES retains its advantages in RTT fairness and delay while preserving near-optimal link utilization across a variety of regions and load conditions.

B. CPU Overhead of BBR-ES

To quantify the runtime overhead of BBR-ES, we measured CPU utilization on the sender host while running a high-load traffic scenario. We started an `iperf` server on a remote machine and launched 24 concurrent TCP flows from the sender for 200 s without bandwidth limitation, using each congestion control algorithm in turn. During each run, we used `mpstat` to sample per-core CPU usage every second and computed the average user+system+softirq CPU utilization over the steady-state interval (discarding the first 10 s of startup).

Table I summarizes the results. All BBR-family algorithms (BBRv1, BBRv3, BBR-R, DA-BBR, and BBR-ES) keep the average per-core CPU utilization below 0.4%. Compared to BBRv1, BBR-ES increases average utilization only slightly, from 0.34% to 0.35% (a difference of about 0.01 percentage points), which is within normal run-to-run variance. This is consistent with our implementation: the existing BBR measurement and pacing infrastructure adds only a small fixed-size history ($N = 8$ samples) and simple trend checks per ACK. For reference, CUBIC shows the highest utilization (about 0.50%) under the same workload. Overall, these results indicate that the runtime CPU overhead of BBR-ES is negligible and comparable to other BBR variants under high-load conditions.

VII. CONCLUSION

This paper introduced BBR-ES, an enhanced congestion control mechanism for RTT-heterogeneous environments. BBR-ES adds an RTT-aware intermediate state after `PROBE_RTT` and a trend-based trigger for entering `PROBE_RTT`, avoiding the one-size-fits-all, global-delay reactions of prior BBR variants. Experiments on both controlled Mininet testbeds and real-world EC2 paths show that BBR-ES achieves higher RTT fairness and link utilization than existing BBR variants, while keeping queuing delay low across diverse RTT combinations, buffer sizes, and flow scales.

BBR-ES still has limitations. Our implementation targets Linux TCP BBR and the experiments focus on single-bottleneck wired paths in Mininet and inter–data center Internet paths on Amazon EC2 with RTTs between about 5 ms and 200 ms. We do not explicitly evaluate wireless, mobile, Wi-Fi, or satellite/NTN environments, where channel variability and path asymmetry can be more pronounced, and the convergence discussion is qualitative rather than a full control-theoretic proof. Extending BBR-ES to these environments and complementing the empirical results with deeper analytical models is left for future work. Nevertheless, our results indicate that model-based congestion control can be made substantially fairer in RTT-heterogeneous networks without sacrificing utilization, and that combining extended state with lightweight trend signals is a promising design pattern for future congestion control in 5G/6G and edge/cloud systems.

REFERENCES

[1] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *Commun. ACM*, vol. 60, no. 2, pp. 58–66, 2017.

[2] *TCP BBR Congestion Control With Amazon Cloudfront*, A. Admin, Seattle, WA, USA, 2019.

[3] A. Cohen, "Netflix is killing my other streaming services," *Compira Labs Blog*, Mar. 4, 2020.

[4] A. Mishra, X. Sun, A. Jain, S. Pande, R. Joshi, and B. Leong, "The great internet TCP congestion control census," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 3, no. 3, pp. 1–24, Jun. 2019.

[5] A. Mishra, "Understanding the modern internet's heterogeneous congestion control landscape," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 52, no. 3, pp. 11–14, Jan. 2025.

[6] A. Mishra, L. Rastogi, R. Joshi, and B. Leong, "Keeping an eye on congestion control in the wild with nebby," in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 136–150.

[7] M. Hock, R. Bless, and M. Zitterbart, "Experimental evaluation of BBR congestion control," in *Proc. IEEE 25th Int. Conf. Netw. Protocols (ICNP)*, Oct. 2017, pp. 1–10.

[8] Z. Han and G. Hasegawa, "Overcoming fairness and latency challenges in BBR with an adaptive delay detection," *IEEE Access*, vol. 13, pp. 37318–37327, 2025.

[9] D. Scholz, B. Jaeger, L. Schwaighofer, D. Raumer, F. Geyer, and G. Carle, "Towards a deeper understanding of TCP BBR congestion control," in *Proc. IFIP Netw. Conf. (IFIP Networking) Workshops*, May 2018, pp. 1–9.

[10] S. Scherrer, M. Legner, A. Perrig, and S. Schmid, "Model-based insights on the performance, fairness, and stability of BBR," in *Proc. 22nd ACM Internet Meas. Conf.*, Oct. 2022, pp. 519–537.

[11] G.-H. Kim and Y.-Z. Cho, "Delay-aware BBR congestion control algorithm for RTT fairness improvement," *IEEE Access*, vol. 8, pp. 4099–4109, 2020.

[12] G.-H. Kim, Y.-J. Song, I. Mahmud, and Y.-Z. Cho, "Enhanced BBR congestion control algorithm for improving RTT fairness," in *Proc. 11th Int. Conf. Ubiquitous Future Netw. (ICUFN)*, Jul. 2019, pp. 358–360.

[13] I. Mahmud and Y.-Z. Cho, "BBR advanced (BBR-A)—Reduced retransmissions with improved fairness," *ICT Exp.*, vol. 6, no. 4, pp. 343–347, Dec. 2020.

[14] K. Kaur, J. Singh, and N. S. Ghumman, "Mininet as software defined networking testing platform," in *Proc. Int. Conf. Commun., Comput. Syst. (ICCCS)*, 2014, pp. 42–139.

[15] "Amazon EC2: Secure and resizable compute capacity for virtually any workload," *Amazon*. [Online]. Available: https://aws.amazon.com/ec2/

[16] N. Cardwell et al., "BBRV3: Algorithm bug fixes and public internet deployment," in *Proc. IETF 117th Meeting*, 2023, pp. 1–36.

[17] M. Welzl, *Network Congestion Control: Managing Internet Traffic*. Hoboken, NJ, USA: Wiley, 2005.

[18] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Syst. Rev.*, vol. 42, no. 5, pp. 64–74, Jul. 2008.

[19] J. Gettys, "Bufferbloat: Dark buffers in the internet," *IEEE Internet Comput.*, vol. 15, no. 3, p. 96, May 2011.

[20] S. Utsumi, J. Ishikawa, F. Ohba, S. M. S. Zabir, and M. Atiquzzaman, "Poster: Toward achieving inter-protocol friendliness through estimation of BBR behaviors using machine learning," in *Proc. ACM Internet Meas. Conf.*, Nov. 2024, pp. 739–740.

[21] Y. Cao, A. Jain, K. Sharma, A. Balasubramanian, and A. Gandhi, "When to use and when not to use BBR: An empirical analysis and evaluation study," in *Proc. Internet Meas. Conf.*, Oct. 2019, pp. 130–136.

[22] S. Ma, J. Jiang, W. Wang, and B. Li, "Fairness of congestion-based congestion control: Experimental evaluation and analysis," 2017, *arXiv:1706.09115*.

[23] M. Yang, P. Yang, C. Wen, Q. Liu, J. Luo, and L. Yu, "Adaptive-BBR: Fine-grained congestion control with improved fairness and low latency," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2019, pp. 1–6.

[24] W. Sun, M. Jia, G. Zhang, and Z. Wang, "RFBBR: A rtt faireness awared algorithm based on BBR," in *Proc. IEEE Int. Conf. Smart Internet Things (SmartIoT)*, Aug. 2020, pp. 124–131.

[25] S. Zheng, J. Liu, X. Yan, Z. Xing, X. Di, and H. Qi, "BBR-R: Improving BBR performance in multi-flow competition scenarios," *Comput. Netw.*, vol. 254, Dec. 2024, Art. no. 110816.

[26] Y. Tao, J. Jiang, S. Ma, L. Wang, W. Wang, and B. Li, "Unraveling the RTT-fairness problem for BBR: A queueing model," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2018, pp. 1–6.

[27] A. Abrol, P. M. Mohan, and T. Truong-Huu, "FaiRTT: An empirical approach for enhanced RTT fairness and bottleneck throughput in BBR," 2024, *arXiv:2403.19973*.

[28] H. Yuan, Y. Han, Y. Zhong, M. Liao, Z. Yuan, and G.-M. Muntean, "FEBBR: A fairness-enhanced approach for BBR congestion control," in *Proc. IEEE Int. Symp. Broadband Multimedia Syst. Broadcast. (BMSB)*, Jun. 2023, pp. 1–6.

[29] C. K. Njogu, W. Yang, H. W. Njogu, and A. Bosire, "BBR-with enhanced fairness (BBR-EFRA): A new enhanced rtt fairness for BBR congestion control algorithm," *Comput. Commun.*, vol. 200, pp. 95–103, Apr. 2023.

[30] G. F. Riley and T. R. Henderson, "The NS-3 network simulator," in *Modeling and Tools for Network Simulation*. Berlin, Germany: Springer, 2010, pp. 15–34.

[31] N. Cardwell et al., "BBRV2: A model-based congestion control performance optimization," in *Proc. IETF 106th Meeting*, 2019, pp. 1–32.

[32] V. Jacobson, "Congestion avoidance and control," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 18, no. 4, pp. 314–329, 1988.

[33] F. Baccelli and D. Hong, "AIMD, fairness and fractal scaling of TCP traffic," in *Proc. 21st Annu. Joint Conf. IEEE Comput. Commun. Societies*, vol. 1, Apr. 2002, pp. 229–238.

[34] R. H. Middleton, C. M. Kellett, and R. N. Shorten, "Fairness and convergence results for additive-increase multiplicative-decrease multiple-bottleneck networks," in *Proc. 45th IEEE Conf. Decis. Control*, Aug. 2006, pp. 1864–1869.

[35] O. Ait-Hellal and E. Altman, "Analysis of TCP Vegas and TCP Reno," *Telecommun. Syst.*, vol. 15, nos. 3–4, pp. 381–404, Dec. 2000.

[36] R. Jain, A. Durresi, and G. Babic, "Throughput fairness index: An explanation," in *Proc. ATM Forum Contribution*, 1999, vol. 99, no. 45, pp. 1–13.